

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Министерство науки и высшего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский авиационный институт

(Национальный исследовательский университет)»

Институт №8

“Компьютерные науки и прикладная математика”

Кафедра 806

Отчет по лабораторной работе №2

по учебной дисциплине “Операционные системы и архитектура компьютера” на тему “Системные вызовы”

Выполнил:

Кекишев А. С.

Проверил:

Ефимов Д.А.

Москва 2026

Содержание

Содержание.....2

Цель исследования 3

Краткое описание архитектуры решения.....4

Схема взаимодействия компонентов 5

Листинг ключевые фрагментов кода.....6

3.1 Чтение из стандартного ввода.....6

3.2 Запуск дочернего процесса 6

3.3 Обработка строк (запись в файл без гласных) 7

3.4 Обработка ошибок.....7

Результат работы 9

4.1 Логи 9

4.2 Скриншоты результатов.....10

Выводы 11

Цель исследования

Цель работы — разработать программу, работающую с системными вызовами в Windows. В ходе выполнения лабораторной работы должны быть получены навыки управления процессами в операционных системах и обеспечения обмена данных между процессами посредством каналов

Краткое описание архитектуры решения

В лабораторной работе использовались один родительский и два дочерних процесса. Родительский процесс читает ввод пользователя из stdin, затем создает файлы, название которых передал ему пользователя, создаёт каналы, запускает дочерние процессы. После чего родительский класс читает ввод пользователем строк, которые с вероятность 0.8 передаёт в первый канал. Дочерние процессы записывают строки пользователя в файл, пропуская (удаляя) все гласные.

Схема взаимодействия компонентов

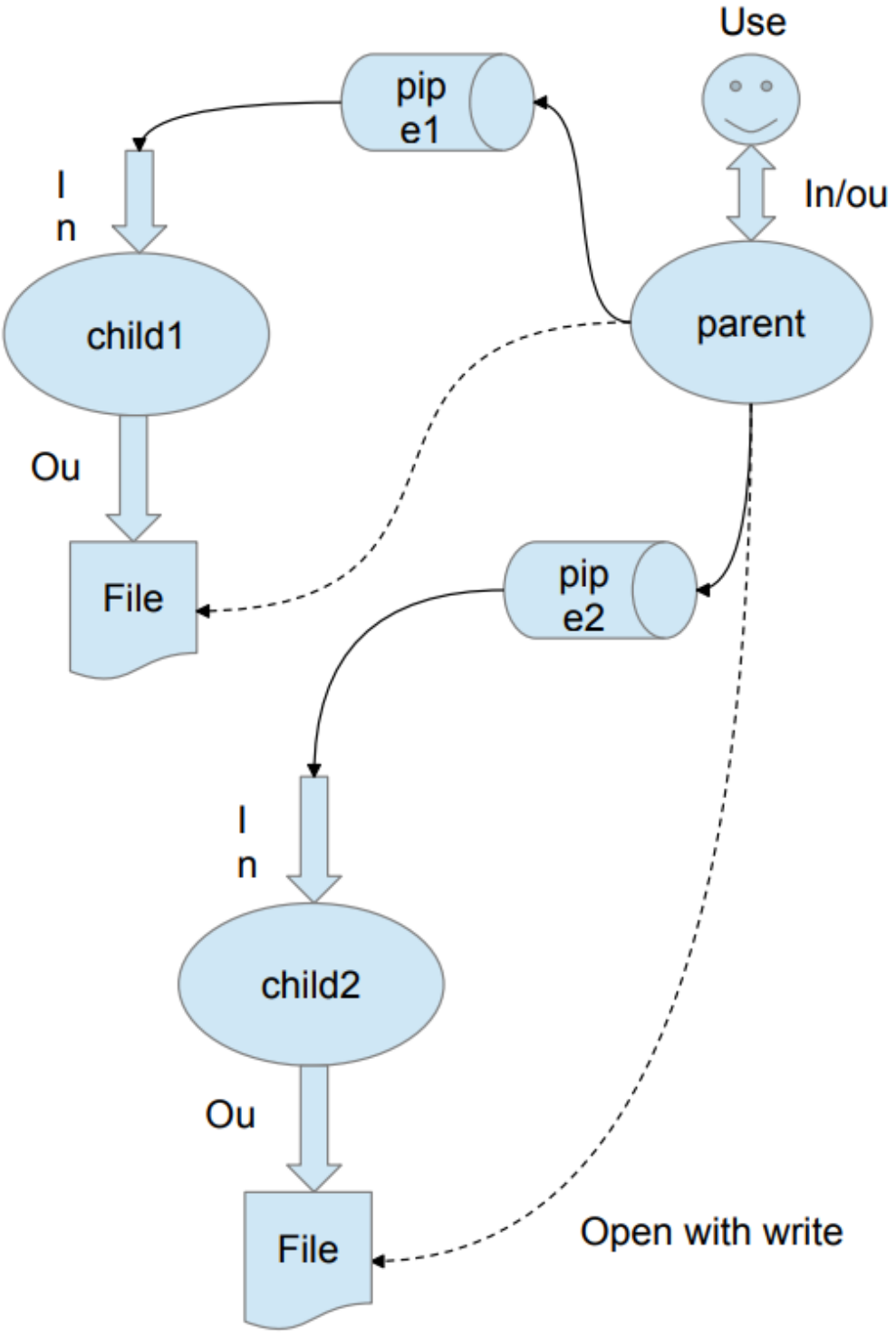


Рисунок 1 – Архитектура решения.

Листинг ключевые фрагментов кода

3.1 Чтение из стандартного ввода

```
      while ((c = fgetc(stdin) while ((c = fgetc(stdin)) != EOF) {
    if (c == '\r') {
        continue;
    }
    if (c == '\n') {
        buf[len] = '\0';
        return buf;
    }

    if (len + 1 >= cap) {
        char* bigger;
        cap *= 2;
        bigger = (char *) realloc(buf, cap);

        if (!bigger) {
            free(buf);
            fail_mem("parent");
            return NULL;
        }

        buf = bigger;
    }

    buf[len++] = (char) c;
}
```

3.2 Запуск дочернего процесса

```
static HANDLE launch_child(const char* label, HANDLE handle_in, HANDLE
handle_out, DWORD* pid) {
    STARTUPINFOA si; // The description of how a new process should start
    PROCESS_INFORMATION pi;
    char cmdline[512];

    set_inherit_descriptor(handle_in,  TRUE);
    set_inherit_descriptor(handle_out, TRUE);

    /**
     * ZeroMemory makes every field be NULL except defined ones.
     * So, Windows applies default settings for them.
     */
    ZeroMemory(&si, sizeof(si));

    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput = handle_in;
    si.hStdOutput = handle_out;
    si.hStdError = GetStdHandle(STD_ERROR_HANDLE);

    ZeroMemory(&pi, sizeof(pi));

    // snprintf fills string with the message
    snprintf(cmdline, sizeof(cmdline), "\"%s\" %s", CHILD_EXE, label);
```

```
    if (!CreateProcessA(NULL, cmdline, NULL, NULL, TRUE, 0, NULL, NULL, &si,
&pi)) {
        fail("CreateProcess (launch " CHILD_EXE ")", "parent");
    }

    set_inherit_descriptor(handle_in,  FALSE);
    set_inherit_descriptor(handle_out, FALSE);

    CloseHandle(pi.hThread);
    *pid = pi.dwProcessId;

    return pi.hProcess;
}
```

3.3 Обработка строк (запись в файл без гласных)

```
static int is_vowel(char c) {
    switch (c) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            return 1;
        default:
            return 0;
    }
}

static void process_line(HANDLE out, const char* line, size_t len) {
    char* result = (char *) malloc(len + 1);
    if (!result) {
        fail_mem("child");
    }

    size_t k = 0;
    for (size_t i = 0; i < len; i++)
        if (!is_vowel(line[i])) {
            result[k++] = line[i];
        }

    result[k++] = '\n';

    write_all(out, result, k, "WriteFile: unsuccessful write to file operation",
"child");
    free(result);
    result = NULL;
}
```

3.4 Обработка ошибок

```
if (WaitForSingleObject(child1, INFINITE) == WAIT_FAILED) {
    fail("WaitForSingleObject (child1)", "parent");
}
if (WaitForSingleObject(child2, INFINITE) == WAIT_FAILED) {
    fail("WaitForSingleObject (child2)", "parent");
}

if (!GetExitCodeProcess(child1, &code1)) {
    fail("GetExitCodeProcess (child1)", "parent");
}
if (!GetExitCodeProcess(child2, &code2)) {
```

```
    fail("GetExitCodeProcess (child2)", "parent");
}
```

Результат работы

4.1 Логи

```
[parent] is running, PID=14040
Enter a name of file for child1: alpha1
Enter a name of file for child2: alpha2
[parent] child1 PID=25248 -> alpha1
[parent] child2 PID=15156 -> alpha2
[parent] Enter lines; The end - Ctrl+Z and Enter
[child2] running, PID=15156
[child1] running, PID=25248
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe1
say some vowels
[parent] line (15 symbols) -> pipe2
say some vowels
```

Рисунок 2 – Логи работы программы.

4.2 Скриншоты результатов

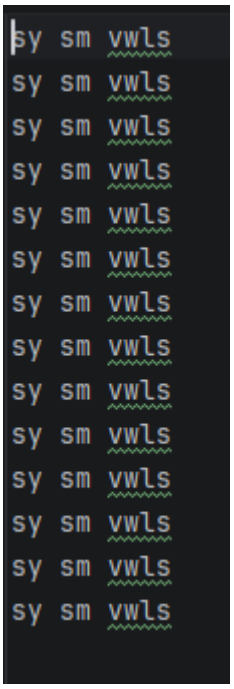


Рисунок 3 – Вывод первого дочернего процесса. Файл alpha1

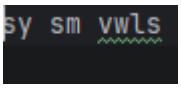


Рисунок 4 – Вывод второго дочернего процесса. Файл alpha2

Выводы

В ходе выполнения лабораторной работы реализована программа, написанная под операционную систему Windows. Программа создаёт родительский и два дочерних процесса, читает пользовательский ввод из стандартного потока, частично делегируя его дочерним процесса и самостоятельно осуществляя обработку ошибок.

Освоены навыки работы с системными вызовами на ОС Windows.

Результат подтверждает разработанная программа, а также освоенные навыки.